Architecture Decision Records Guide

Architecture Decision Records (ADR) Guide

What is an ADR?

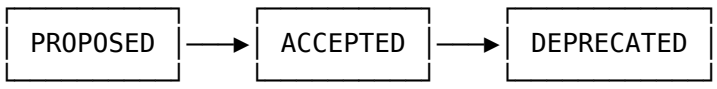
An **Architecture Decision Record (ADR)** is a document that captures an important architectural decision made along with its context and consequences. ADRs are stored in the **Governance Log** within the Architecture Repository.

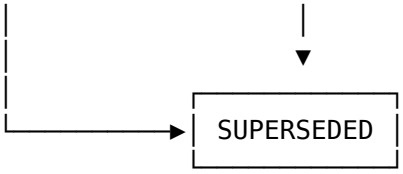
ADR Structure

Every ADR should contain these key sections:

Section	Description
ID	Unique identifier (e.g., ADR-001)
Title	Short descriptive name
Status	Current state of the decision
Date	When the decision was made
Context	Why this decision is needed
Decision	What was decided
Rationale	Why this option was chosen
Alternatives	Other options considered
Consequences	Impact of this decision
Stakeholders	Who was involved
Related ADRs	Links to related decisions

ADR Status Lifecycle





Status	Meaning
Proposed	Under review, not yet approved
Accepted	Approved and in effect
Deprecated	No longer recommended
Superseded	Replaced by another ADR

Sample ADR Documents

ADR-001: Microservices Architecture Pattern

Field	Value
ADR ID	ADR-001
Title	Adoption of Microservices Architecture
Status	✅ ACCEPTED
Date	2024-03-15
Author	Enterprise Architecture Team
Review Board	Architecture Review Board (ARB)

Context

Our current monolithic e-commerce platform has the following challenges: - **Scalability Issues**: Cannot scale individual components independently - **Deployment Risk**: Single deployment for entire application - **Technology Lock-in**: Entire system tied to Java/Spring - **Team Bottlenecks**: Multiple teams working on same codebase - **Release Velocity**: 4-week release cycles due to coordination overhead

Decision

We will adopt a microservices architecture pattern for the next-generation platform with the following characteristics:

1. **Domain-Driven Design** - Services aligned to business domains
2. **API-First Approach** - REST/gRPC for inter-service communication
3. **Database per Service** - Each service owns its data
4. **Event-Driven** - Async communication via message queues
5. **Container-Based** - Kubernetes for orchestration

Rationale

Criterion	Monolith	Microservices	Winner
Independent Scaling	❌	✅	Microservices
Deployment Risk	High	Low	Microservices
Technology Flexibility	❌	✅	Microservices
Team Autonomy	❌	✅	Microservices
Operational Complexity	Low	High	Monolith
Initial Development Speed	✅	❌	Monolith

Decision: Benefits outweigh complexity for our scale (50+ developers, 10M+ users)

Alternatives Considered

Option A: Modular Monolith

- **Pros**: Simpler operations, easier debugging
- **Cons**: Still coupled deployments, limited scaling
- **Verdict**: Rejected - doesn't solve scaling needs

Option B: Serverless (FaaS)

- **Pros**: No infrastructure management, auto-scaling
- **Cons**: Cold starts, vendor lock-in, debugging challenges
- **Verdict**: Rejected - not suitable for stateful workflows

Option C: Microservices (Selected)

- **Pros**: Independent scaling, team autonomy, tech flexibility
- **Cons**: Operational complexity, distributed systems challenges

- **Verdict**: Selected - best fit for organization size and needs

Consequences

Positive Consequences

- ✅ Independent deployment of services (CI/CD per service)
- ✅ Horizontal scaling per business domain
- ✅ Technology diversity (Python for ML, Go for high-perf, Node for APIs)
- ✅ Team autonomy and ownership
- ✅ Fault isolation (one service failure doesn't crash entire system)

Negative Consequences

- ⚠️ Increased operational complexity
- ⚠️ Need for service mesh and API gateway
- ⚠️ Distributed tracing and logging infrastructure required
- ⚠️ Data consistency challenges (eventual consistency)
- ⚠️ Higher initial infrastructure costs

Mitigation Strategies

Risk	Mitigation
Operational Complexity	Invest in DevOps team, use managed Kubernetes
Distributed Debugging	Implement Jaeger/Zipkin tracing
Data Consistency	Saga pattern, event sourcing
Service Discovery	Use Consul or Kubernetes DNS

Stakeholders

Role	Name	Decision
Chief Architect	John Smith	✅ Approved
VP Engineering	Sarah Johnson	✅ Approved
Security Architect	Mike Chen	✅ Approved with conditions
Operations Lead	Lisa Wong	⚠️ Approved with concerns

Related ADRs

- ADR-002: API Gateway Selection

- ADR-003: Message Queue Technology
- ADR-005: Database Strategy

ADR-002: API Gateway Selection

Field	Value
ADR ID	ADR-002
Title	Kong as Enterprise API Gateway
Status	✅ ACCEPTED
Date	2024-03-22
Author	Platform Team
Supersedes	None

Context

With the adoption of microservices (ADR-001), we need a centralized API Gateway for: - Request routing and load balancing - Authentication and authorization - Rate limiting and throttling - API versioning - Monitoring and analytics

Decision

We will use Kong Enterprise as our API Gateway platform.

Alternatives Considered

Gateway	Pros	Cons	Score
Kong	Open-source core, plugin ecosystem, Kubernetes-native	Enterprise features require license	8/10
AWS API Gateway	Managed, serverless integration	Vendor lock-in, limited customization	6/10
Apigee	Strong analytics, developer portal	Expensive, complex setup	5/10
NGINX Plus	High performance, familiar	Limited built-in features	6/10

Consequences

Positive

- ✅ Rich plugin ecosystem (50+ plugins)
- ✅ Kubernetes-native with Ingress Controller
- ✅ Open-source core reduces vendor lock-in
- ✅ Strong community support

Negative

- ⚠️ Enterprise license cost for advanced features
- ⚠️ Learning curve for team

Related ADRs

- ADR-001: Microservices Architecture Pattern

ADR-003: Event-Driven Architecture with Kafka

Field	Value
ADR ID	ADR-003
Title	Apache Kafka for Event Streaming
Status	✅ ACCEPTED
Date	2024-04-01
Author	Data Platform Team

Context

Microservices require asynchronous communication for: - Decoupling services - Event sourcing - Real-time analytics - Audit logging

Decision

We will use Apache Kafka as our event streaming platform with Confluent Cloud for managed operations.

Alternatives Considered

Technology	Throughput	Ordering	Persistence	Verdict
Kafka	Very High	✅ Partition-level	✅ Configurable	Selected
RabbitMQ	High	❌ Limited	⚠️ Optional	Rejected
AWS SQS/SNS	High	❌ FIFO limited	✅ Yes	Rejected
Redis Streams	Very High	✅ Yes	⚠️ Memory-based	Rejected

Consequences

Positive

- ✅ High throughput (millions of events/second)
- ✅ Event replay capability
- ✅ Strong ordering guarantees per partition
- ✅ Ecosystem (Kafka Connect, ksqlDB, Schema Registry)

Negative

- ⚠️ Operational complexity (if self-managed)
- ⚠️ Learning curve for developers
- ⚠️ Cost of Confluent Cloud

ADR-004: Authentication Strategy

Field	Value
ADR ID	ADR-004
Title	OAuth 2.0 + JWT for Authentication
Status	✅ ACCEPTED
Date	2024-04-10
Author	Security Team

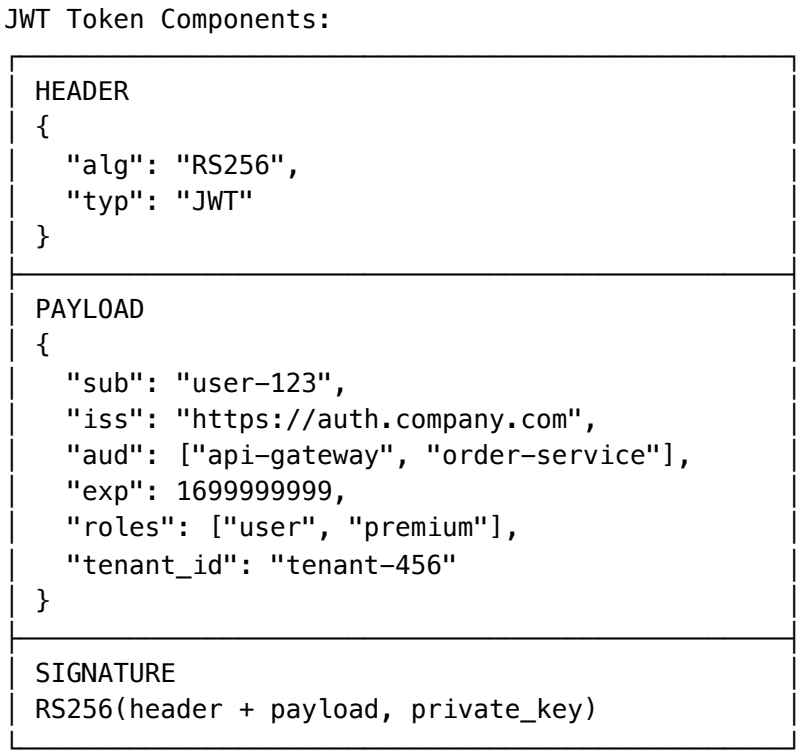
Context

Need standardized authentication across: - Public APIs (mobile, web) - Internal service-to-service - Third-party integrations

Decision

We will implement OAuth 2.0 with JWT tokens using Keycloak as the Identity Provider.

Token Structure



Consequences

Positive

- ✅ Stateless authentication (no session storage)
- ✅ Industry standard (OAuth 2.0)
- ✅ Fine-grained access control via scopes
- ✅ Support for multiple identity providers

Negative

- ⚠️ Token revocation complexity
- ⚠️ Token size can be large with many claims

ADR-005: Database per Service Strategy

Field	Value
ADR ID	ADR-005
Title	Polyglot Persistence - Database per Service
Status	✅ ACCEPTED
Date	2024-04-15
Author	Data Architecture Team

Context

Microservices require data autonomy. Each service should: - Own its data schema - Choose appropriate database technology - Scale independently

Decision

Each microservice will own its database, with technology selection based on use case.

Database Selection Matrix

Service	Data Pattern	Database	Justification
User Service	Relational, ACID	PostgreSQL	Strong consistency for user data
Product Catalog	Document, flexible schema	MongoDB	Variable product attributes
Order Service	Relational, transactions	PostgreSQL	ACID for financial data
Search Service	Full-text search	Elasticsearch	Fast text search
Session Store	Key-value, high speed	Redis	Low latency reads
Analytics	Time-series, aggregations	ClickHouse	Fast analytical queries
Audit Log	Append-only, immutable	Kafka + S3	Event sourcing

Consequences

Positive

- ✅ Right tool for the job
- ✅ Independent scaling

- ✅ Team autonomy
- ✅ No shared database bottleneck

Negative

- ⚠️ No cross-service joins (use API composition)
- ⚠️ Eventual consistency between services
- ⚠️ Multiple database technologies to manage

ADR-006: Frontend Architecture (Deprecated)

Field	Value
ADR ID	ADR-006
Title	React SPA Architecture
Status	🚫 DEPRECATED
Date	2024-02-01
Deprecated Date	2024-08-15
Superseded By	ADR-010

Context

Initial decision for single-page application architecture.

Decision

Use React with Create React App for frontend.

Reason for Deprecation

- SEO requirements emerged
- Performance issues with large bundles
- Better alternatives available (Next.js)

See ADR-010 for current frontend strategy.

ADR Template

Use this template for new ADRs:

```
# ADR-[NUMBER]: [TITLE]

| Field | Value |
|-------|-------|
| **ADR ID** | ADR-XXX |
| **Title** | [Short descriptive title] |
| **Status** | PROPOSED / ACCEPTED / DEPRECATED / SUPERSEDED |
| **Date** | YYYY-MM-DD |
| **Author** | [Team/Person] |
| **Supersedes** | [ADR-XXX if applicable] |

## Context

[What is the issue? Why do we need to make this decision?]

## Decision

[What is the change that we're proposing and/or doing?]

## Rationale

[Why was this option chosen over alternatives?]

## Alternatives Considered

### Option A: [Name]
- **Pros**: ...
- **Cons**: ...
- **Verdict**: Rejected/Selected

### Option B: [Name]
...

## Consequences

### Positive
- ✅ [Benefit 1]
- ✅ [Benefit 2]

### Negative
- ⚠️ [Drawback 1]
- ⚠️ [Drawback 2]

### Mitigation
| Risk | Mitigation |
|------|------------|
| [Risk 1] | [Strategy] |

## Stakeholders

| Role | Name | Decision |
|------|------|----------|
| [Role] | [Name] | Approved/Rejected |
```

```
## Related ADRs
- ADR-XXX: [Title]
```

ADR Best Practices

Do's ✅

1. **Keep it concise** - Focus on the decision, not implementation details
2. **Document alternatives** - Show you considered other options
3. **Include consequences** - Both positive and negative
4. **Link related ADRs** - Build a decision network
5. **Update status** - Mark deprecated/superseded decisions
6. **Version control** - Store ADRs in git alongside code

Don'ts ❌

1. **Don't delete ADRs** - Mark as deprecated instead
2. **Don't make them too long** - Keep under 2 pages
3. **Don't skip context** - Future readers need background
4. **Don't ignore stakeholders** - Document who approved

ADR in TOGAF Context

TOGAF Phase	ADR Usage
Phase A	Strategic technology direction decisions
Phase B	Business capability decisions
Phase C	Application/data architecture decisions
Phase D	Technology platform decisions
Phase E	Solution approach decisions
Phase F	Migration strategy decisions
Phase G	Governance and standards decisions

Storage Location

ADRs are stored in the **Governance Log** within the **Architecture Repository**.

```
Architecture Repository
├── Architecture Metamodel
├── Architecture Capability
├── Architecture Landscape
├── Standards Information Base
├── Reference Library
└── Governance Log  ◄── ADRs stored here
    ├── ADR-001.md
    ├── ADR-002.md
    ├── ADR-003.md
    └── ...
```

Interview Tips 💡

1. **"What is an ADR?"** > A document capturing architectural decisions with context, rationale, and consequences
2. **"Where are ADRs stored in TOGAF?"** > In the Governance Log within the Architecture Repository
3. **"What's the difference between ADR and RFC?"** > RFC proposes standards for comment; ADR documents a specific decision made
4. **"Can ADRs be changed?"** > They should be immutable - mark as deprecated and create new ADR if needed
5. **"Who creates ADRs?"** > Architects, reviewed by Architecture Review Board (ARB)